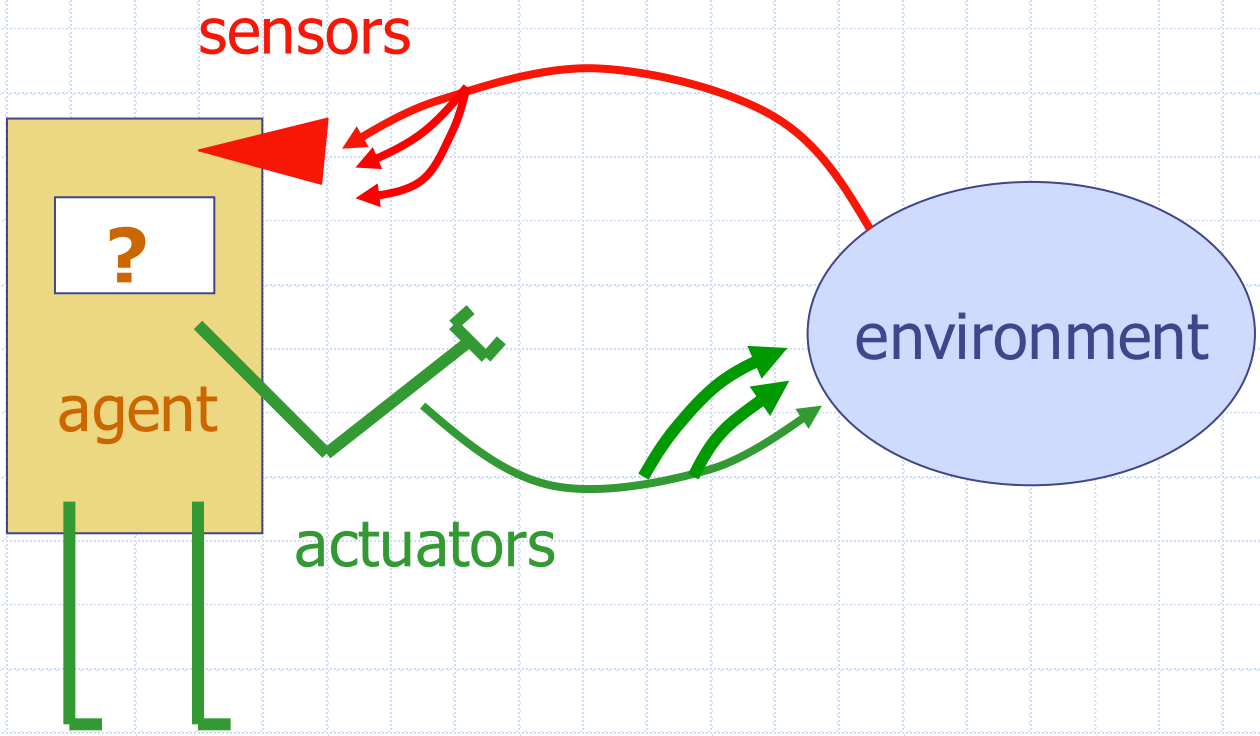
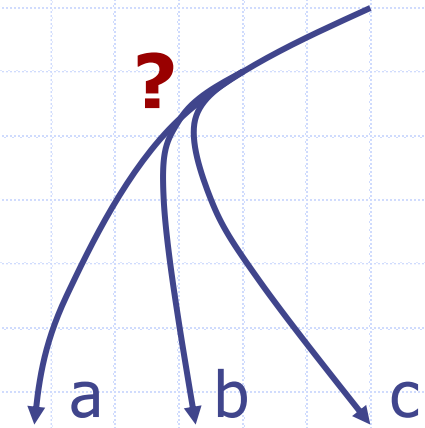
Decision Making Under Uncertainty

Russell and Norvig: ch 16, 17

Utility-Based Agent



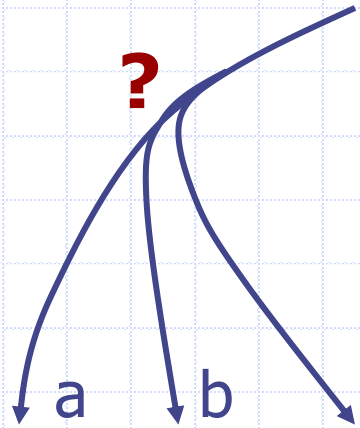
Non-deterministic vs. Probabilistic Uncertainty



{a,b,c}

→ decision that is best for worst case

Non-deterministic model



{a(p_a),b(p_b),c(p_c)}

→ decision that maximizes expected utility value

Probabilistic model

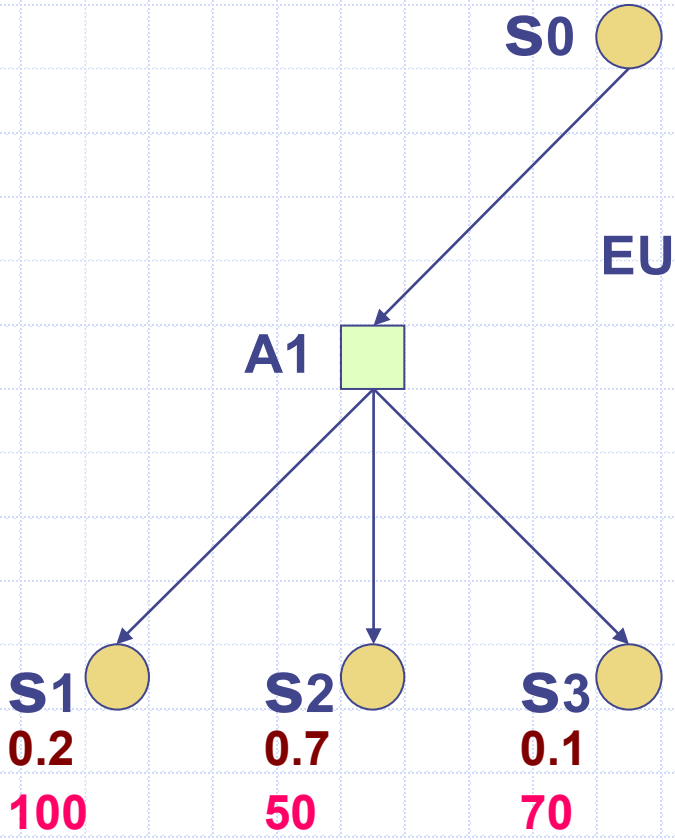
~ Adversarial search

Expected Utility

- Random variable X with n values $x_1,\ldots,x_n$ and distribution $(p_1,\ldots,p_n)$
  E.g.: X is the state reached after doing an action A under uncertainty
- Function U of X
  E.g., U is the utility of a state
- The expected utility of A is

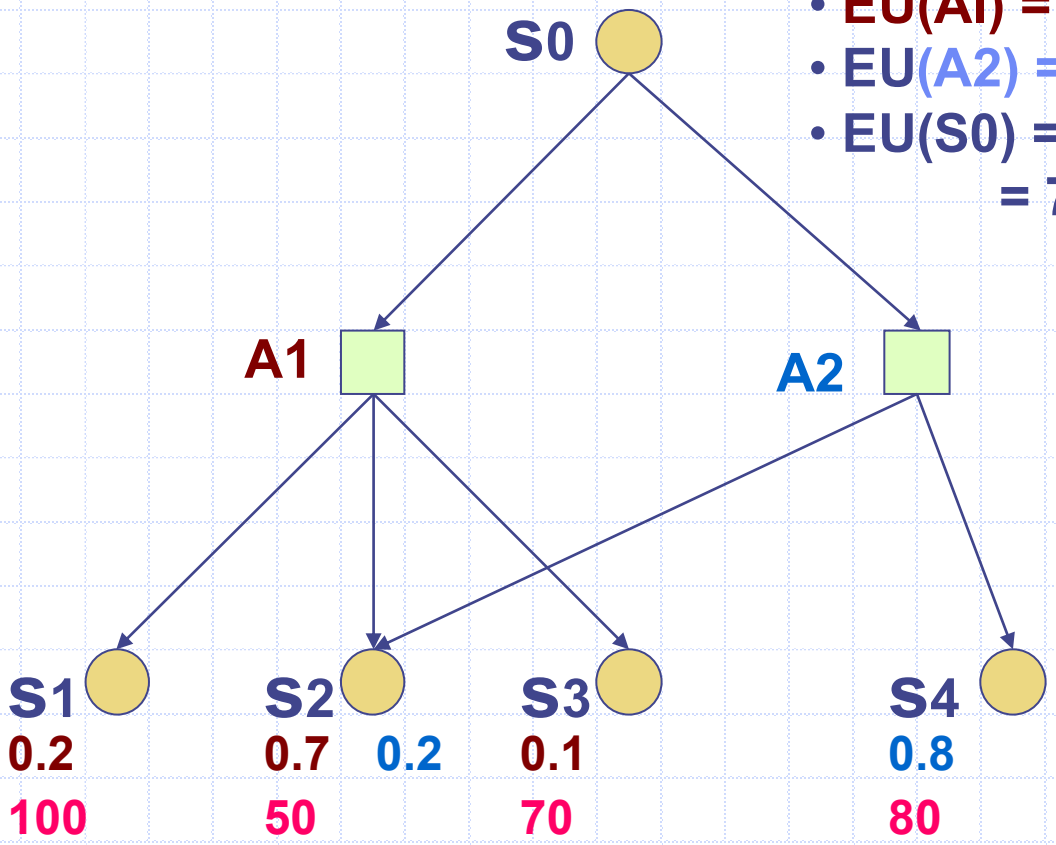
$$EU[A] = \sum_{i=1,\ldots,n} p(x_i|A)U(x_i)$$

One State/One Action Example



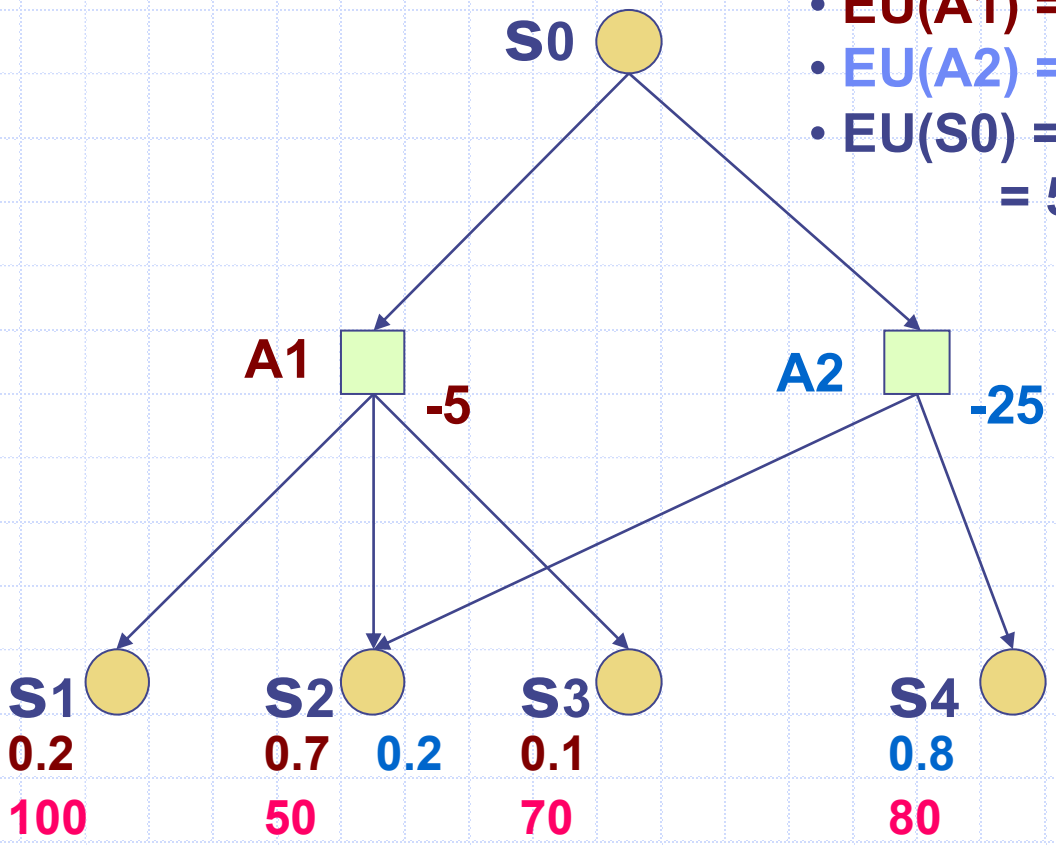
EU(A1) = 100 x 0.2 + 50 x 0.7 + 70 x 0.1
= 20 + 35 + 7
= 62

One State/Two Actions Example



- EU(AI) = 62
- EU(A2) = 74
- EU(S0) = max{EU(A1),EU(A2)}
  = 74

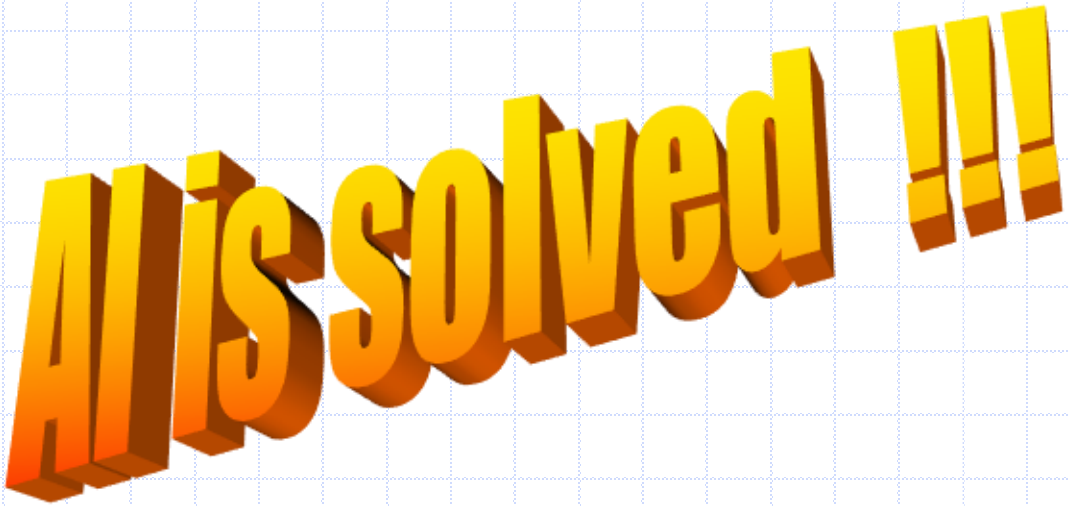
Introducing Action Costs



- EU(A1) = 62 – 5 = 57
- EU(A2) = 74 – 25 = 49
- EU(S0) = max{EU(A1),EU(A2)}
  = 57

MEU Principle

- rational agent should choose the action that maximizes agent's expected utility
- this is the basis of the field of decision theory
- normative criterion for rational choice of action



Not quite…

- Must have **complete** model of:
  - Actions
  - Utilities
  - States
- Even if you have a complete model, will be computationally **intractable**
- In fact, a truly rational agent takes into account the utility of reasoning as well---**bounded rationality**
- Nevertheless, great progress has been made in this area recently, and we are able to solve much more complex decision theoretic problems than ever before

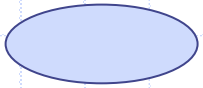
We'll look at

- Decision Theoretic Planning
  - Simple decision making (ch. 16)
  - Sequential decision making (ch. 17)

Decision Networks

- Extend BNs to handle actions and utilities
- Also called Influence diagrams
- Make use of BN inference
- Can do Value of Information calculations

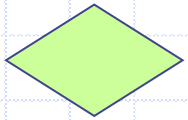
Decision Networks cont.



- Chance nodes: random variables, as in BNs

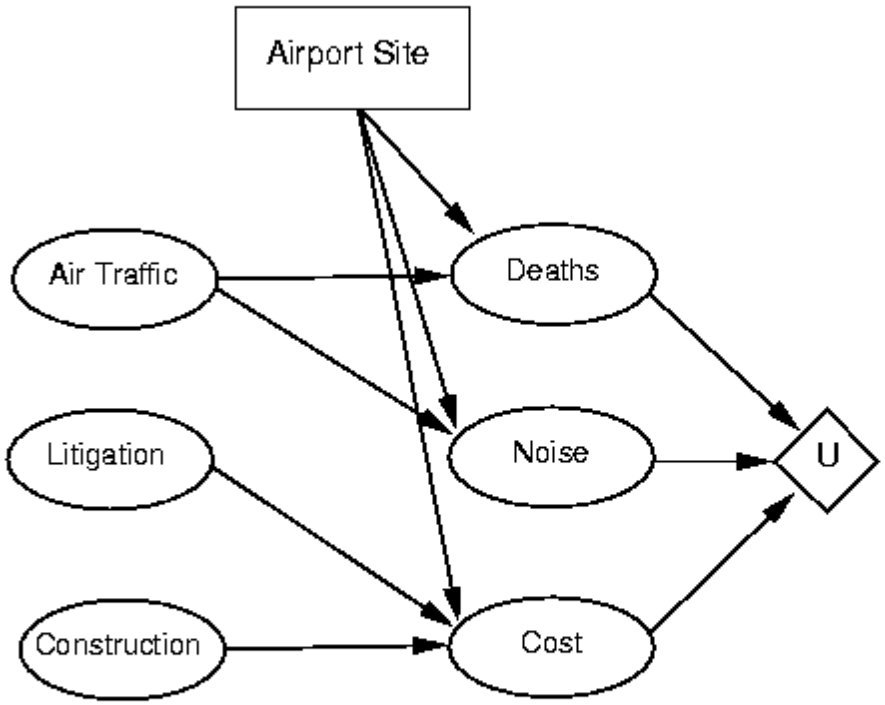


- Decision nodes: actions that decision maker can take

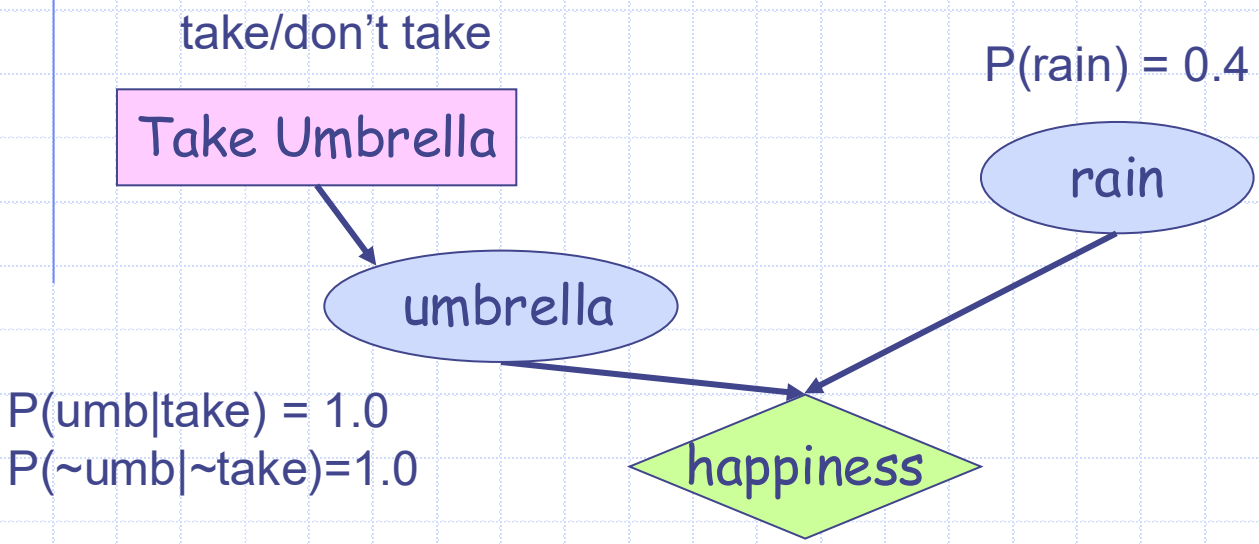


- Utility/value nodes: the utility of the outcome state.

R&N example


Airport Site
Air Traffic
Litigation
Construction
Deaths
Noise
Cost
U

Umbrella Network

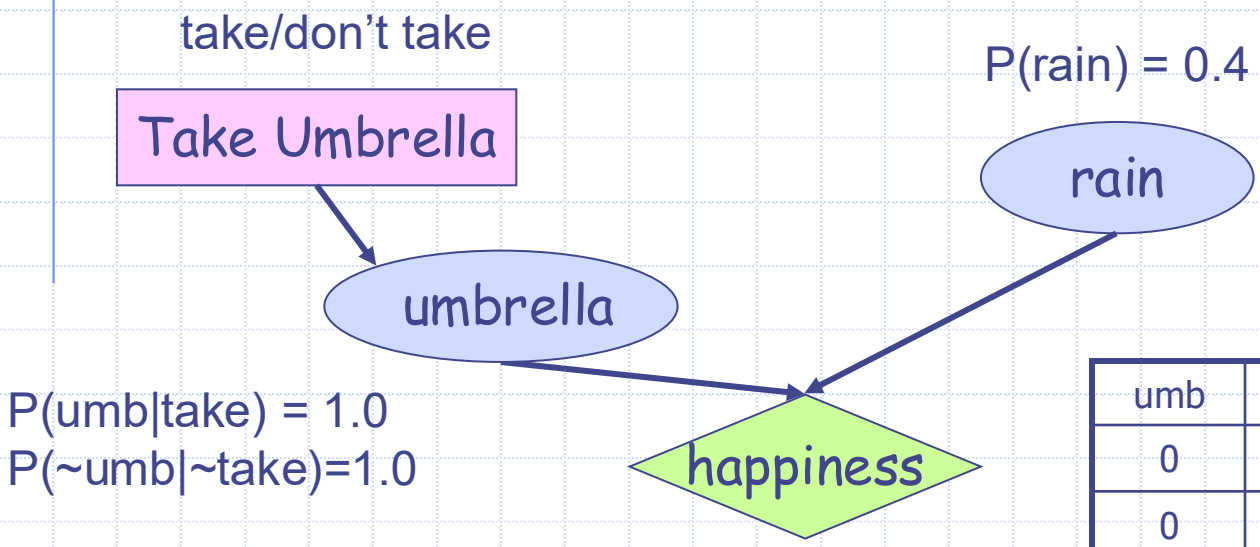


U(~umb, ~rain) = 100
U(~umb, rain) = -100
U(umb,~rain) = 0
U(umb,rain) = -25

Evaluating Decision Networks

- Set the evidence variables for current state
- For each possible value of the decision node:
  - Set decision node to that value
  - Calculate the posterior probability of the parent nodes of the utility node, using BN inference
  - Calculate the resulting utility for action
- return the action with the highest utility

Umbrella Network



P(umb|take) = 1.0
P(~umb|~take)=1.0

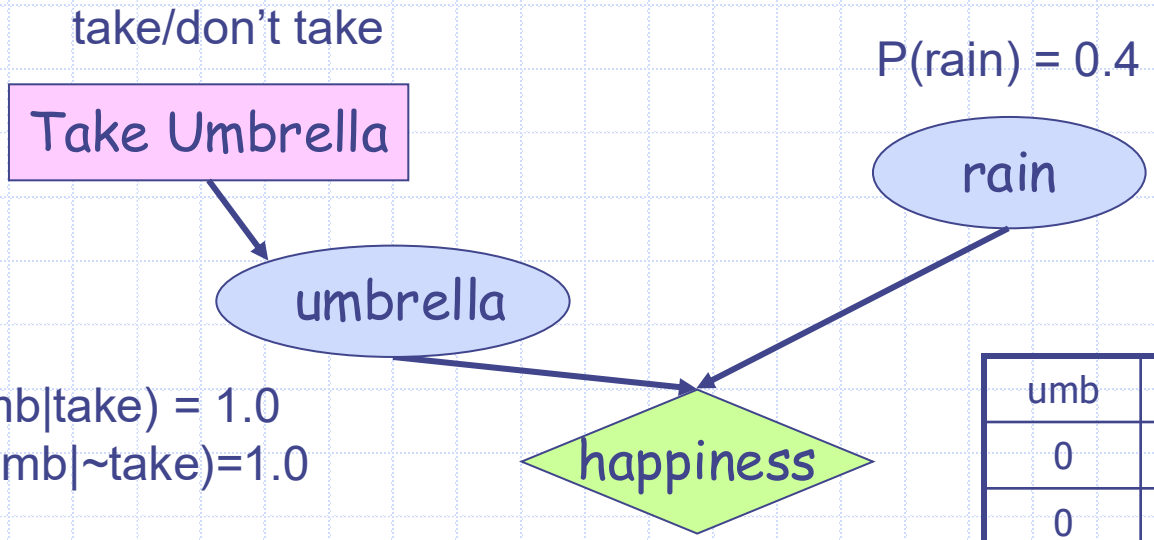
U(~umb, ~rain) = 100
U(~umb, rain) = -100
U(umb,~rain) = 0
U(umb,rain) = -25

#1

umb	rain	P(umb,rain | take)
0	0	0
0	1	0
1	0	0.6
1	1	0.4

#1: EU(take) = -10

Umbrella Network



P(umb|take) = 1.0
P(~umb|~take)=1.0

U(~umb, ~rain) = 100
U(~umb, rain) = -100
U(umb,~rain) = 0
U(umb,rain) = -25

#2

umb	rain	P(umb,rain | ~take)
0	0	0.6
0	1	0.4
1	0	0
1	1	0

#2: EU(~take) = 20

Making Decision

- Choose the action with the highest utility

- #1: EU(take) = -10
- #2: EU(~take) = 20

- Therefore : The decision is **~take**.

Value of Perfect Information (VPI)

- **How much is it worth to observe (with certainty) a random variable X?**
  - Suppose the agent's current knowledge is E. The value of the current best action α is:
    $EU(\alpha \mid E) = \max_A \sum_i U(Result_i(A))\, p(Result_i(A) \mid E, Do(A))$
  - The value of the new best action *after observing the value of X* is:
    $EU(\alpha' \mid E,X) = \max_A \sum_i U(Result_i(A))\, p(Result_i(A) \mid E, X, Do(A))$
- ...But we don't know the value of X yet, so we have to sum over its possible values
- The value of perfect information for X is therefore:

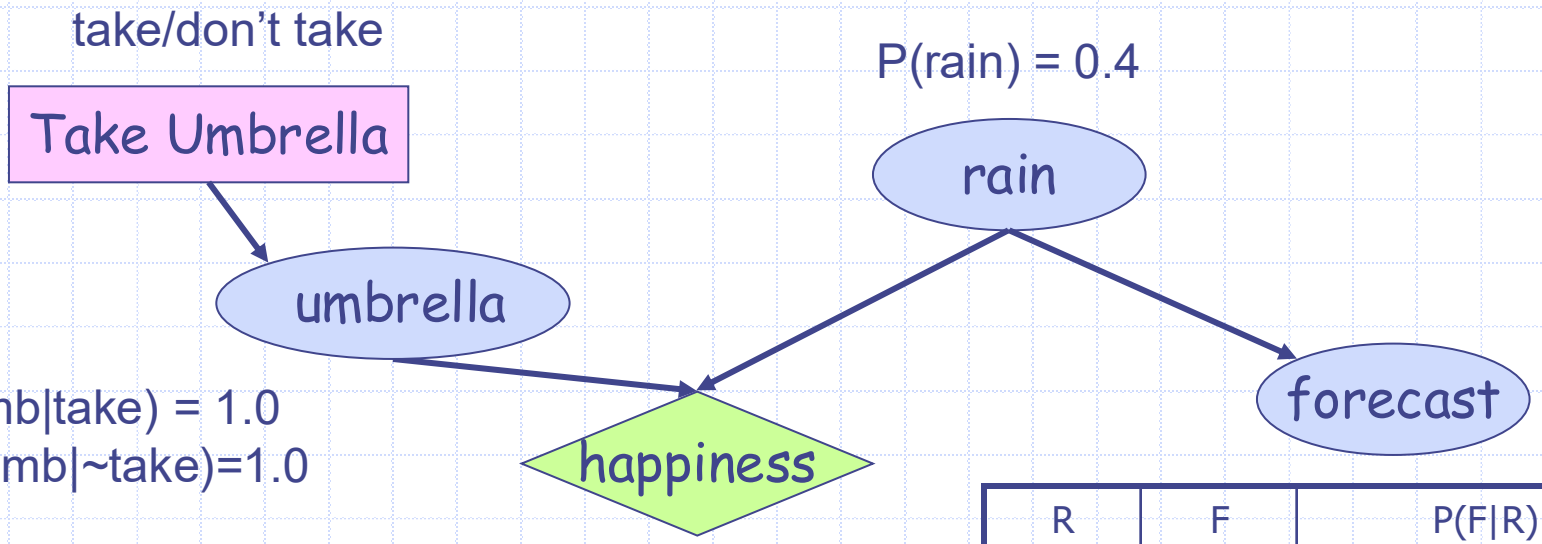
$$VPI(X) = \left(\sum_k p(x_k \mid E)\, EU(\alpha_{xk} \mid x_k, E) \right) - EU(\alpha \mid E)$$

Probability of each value of X

Expected utility of the best action given that value of X

Expected utility of the best action if we don't know X (i.e., currently)

Umbrella Network



P(umb|take) = 1.0
P(~umb|~take)=1.0

U(~umb, ~rain) = 100
U(~umb, rain) = -100
U(umb,~rain) = 0
U(umb,rain) = -25

R	F	P(F|R)
0	0	0.8
0	1	0.2
1	0	0.3
1	1	0.7

Evidence P(R|F)

Find P(F)

- **P(F=Rainy)** = P(F=Rainy|R=Rain)P(R=Rain) + P(F=Rainy|R=~Rain)P(R=~Rain)

  P(F=1) = P(F=1|R=1)P(R=1) + P(F=1|R=0)P(R=0)

  = (0.7) (0.4) + (0.2) (0.6) = 0.28 + 0.12 = **0.4**

- **P(F=~Rainy)** = P(F=~Rainy|R=Rain)P(R=Rain) + P(F=~Rainy|R=~Rain)P(R=~Rain)

  P(F=0) = P(F=0|R=1)P(R=1) + P(F=0|R=0)P(R=0)

  = (0.3) (0.4) + (0.8) (0.6) = 0.12 + 0.48 = **0.6**

Now, you can find P(R|F)

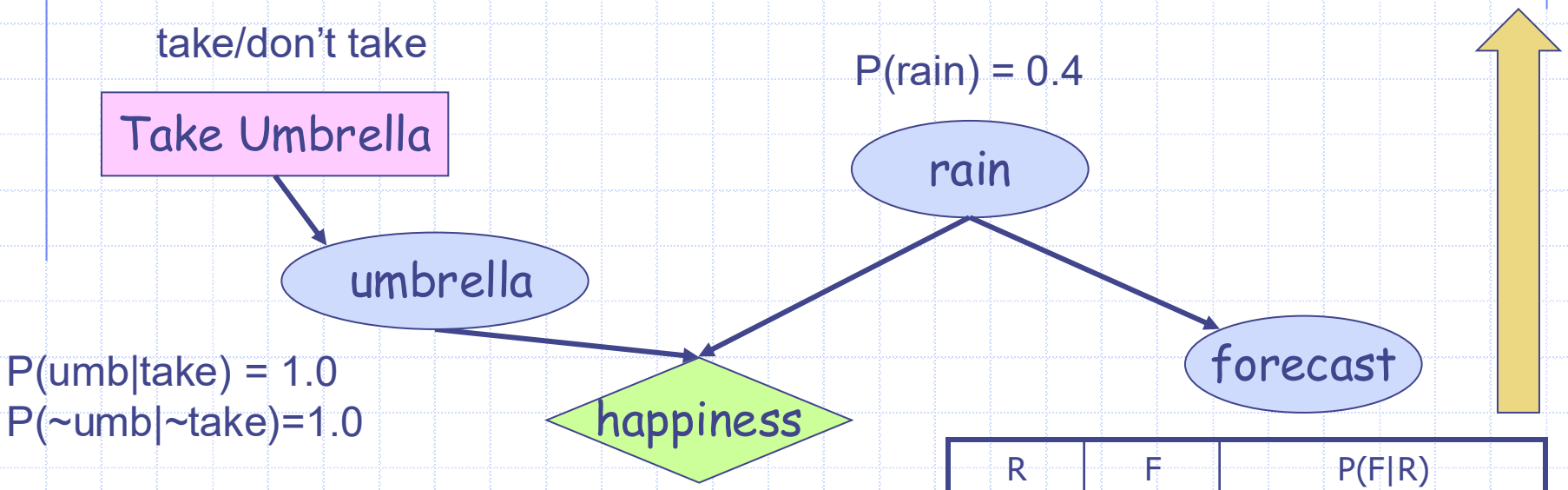
- P(R|F) = P(F|R)P(R)/P(F)

F	R	P(R|F)
0	0	= (0.8)(0.6)/(0.6) = **0.8**
0	1	= (0.3)(0.4)/(0.6) = **0.2**
1	0	= (0.2)(0.6)/(0.4) = **0.3**
1	1	= (0.7)(0.4)/(0.4) = **0.7**

Umbrella Network

P(F=rainy) = 0.4

F	R	P(R|F)
0	0	0.8
0	1	0.2
1	0	0.3
1	1	0.7



P(umb|take) = 1.0
P(~umb|~take)=1.0

U(~umb, ~rain) = 100
U(~umb, rain) = -100
U(umb,~rain) = 0
U(umb,rain) = -25

R	F	P(F|R)
0	0	0.8
0	1	0.2
1	0	0.3
1	1	0.7

umb	rain	P(umb,rain | take, rainy)
0	0	0
0	1	0
1	0	0.3
1	1	0.7

#1: EU(take|rainy) = -17.5

umb	rain	P(umb,rain | take, ~rainy)
0	0	0
0	1	0
1	0	0.8
1	1	0.2

#3: EU(take|~rainy) = -5

umb	rain	P(umb,rain | ~take, rainy)
0	0	0.3
0	1	0.7
1	0	0
1	1	0

#2: EU(~take|rainy) = -40

umb	rain	P(umb,rain |~take, ~rainy)
0	0	0.8
0	1	0.2
1	0	0
1	1	0

#4: EU(~take|~rainy) = 60

What is the optimal decision?

- **Forecast = ~Rain & Umbrella = ~take**
  - EU(~take|F=0) = $\sum_R P(R \mid F=0)U(\sim \text{take}, \text{R})$
  - (0.8)(100) + (0.2)(-100) = **60**
- **Forecast = Rain & Umbrella = ~take**
  - EU(~take|F=1) = $\sum_R P(R \mid F=1)U(\sim \text{take}, \text{R})$
  - (0.3)(100) + (0.7)(-100) = **-40**
- **Forecast = ~Rain & Umbrella = take**
  - EU(take|F=0) = $\sum_R P(R \mid F=0)U(\text{take}, \text{R})$
  - (0.8)(0) + (0.2)(-25) = **-5**
- **Forecast = Rain & Umbrella = take**
  - EU(take|F=1) = $\sum_R P(R \mid F=1)U(\text{take}, \text{R})$
  - (0.3)(0) + (0.7)(-25) = **-17.5**

Optimal Decisions

- **No evidence**
- MEU() = $\max_{\alpha}$ EU(α) = 20 α = ~take

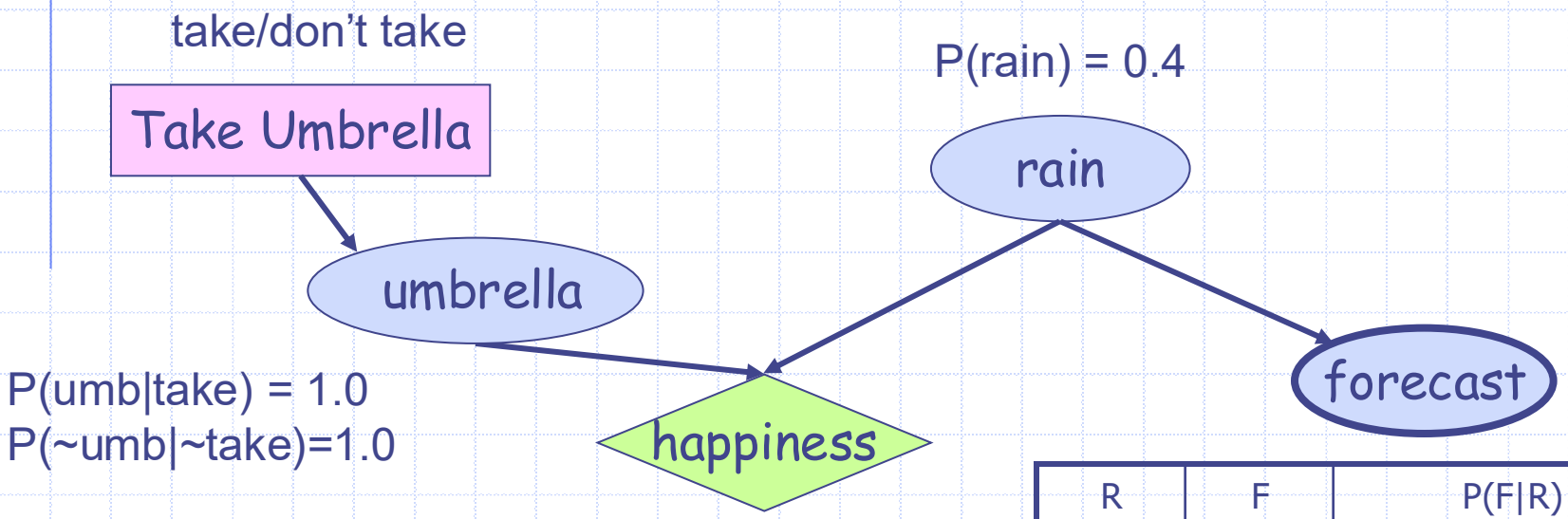
- **If we have F=rainy**
- MEU(F=rainy) = $\max_{\alpha}$ EU(α|F=rainy)
  = -17.5 α = take

- **If we have F=~rainy**
- MEU(F=~rainy) = $\max_{\alpha}$ EU(α|F=~rainy)
  = 60 α = ~take

Umbrella Network

P(F=rainy) = 0.4

F	R	P(R|F)
0	0	0.8
0	1	0.2
1	0	0.3
1	1	0.7



P(umb|take) = 1.0
P(~umb|~take)=1.0

U(~umb, ~rain) = 100
U(~umb, rain) = -100
U(umb,~rain) = 0
U(umb,rain) = -25

R	F	P(F|R)
0	0	0.8
0	1	0.2
1	0	0.3
1	1	0.7

VPI (Value of Information)

- **VPI(forecast)**=
  $P(F=\text{rainy})EU(\alpha_{F=\text{rainy}}) +$
  $P(F=\sim\text{rainy})EU(\alpha_{F=\sim\text{rainy}}) -$
  $EU(\alpha)$

$= (0.4)(-17.5) + (0.6)(60) - (20)$

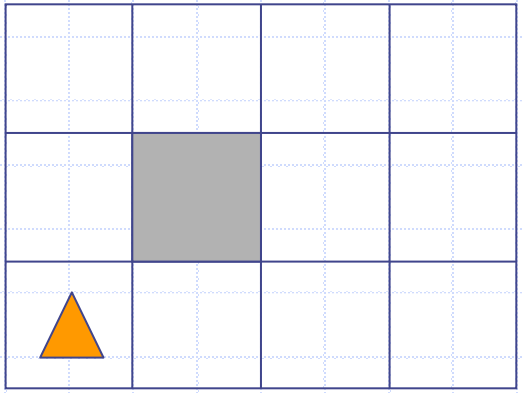
$= -7 + 36 - 20$

$= 9$

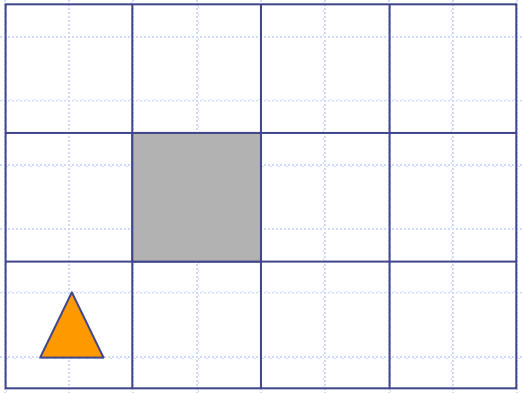
Sequential Decision Making

- Finite Horizon
  - there is a fixed time N
  - So, with a finite horizon, the optimal action in a given state could change over time.
  - We say that the optimal policy for a finite horizon is **nonstationary**.
- Infinite Horizon
  - With no fixed time limit
  - the optimal action depends only on the current state
  - the optimal policy is **stationary**.

Simple Robot Navigation Problem



Probabilistic Transition Model

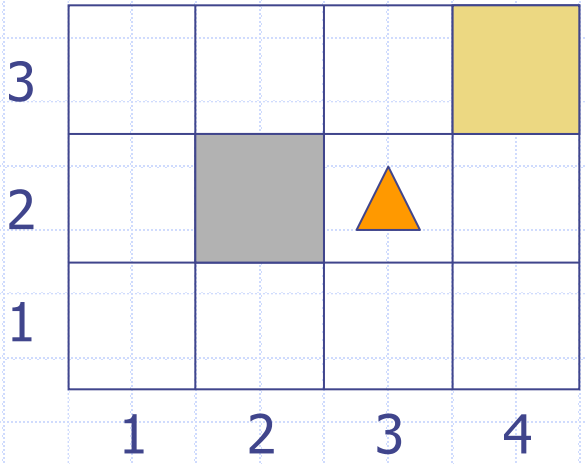


- In each state, the possible actions are U, D, R, and L
- The effect of U is as follows (transition model):
  - With probability 0.8 the robot moves up one square (if the robot is already in the top row, then it does not move)
  - With probability 0.1 the robot moves right one square (if the robot is already in the rightmost row, then it does not move)
  - With probability 0.1 the robot moves left one square (if the robot is already in the leftmost row, then it does not move)

Markov Property

The transition properties depend only on the current state, not on previous history (how that state was reached)

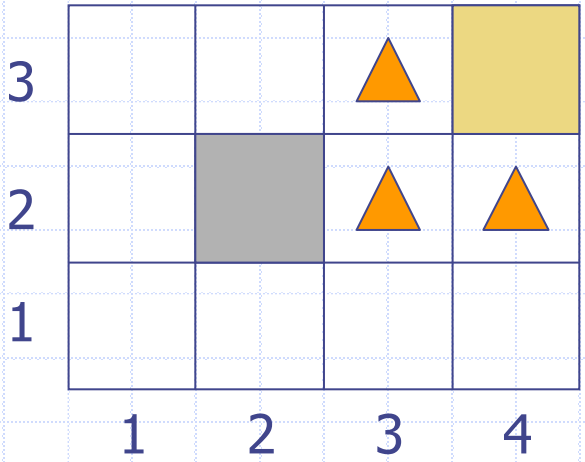
Sequence of Actions

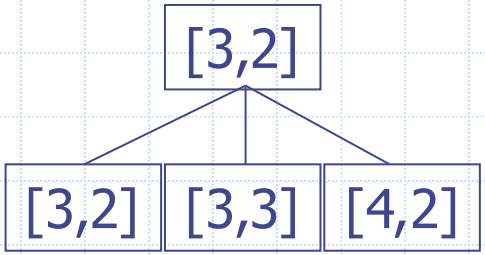


[3,2]

- Planned sequence of actions: (U, R)

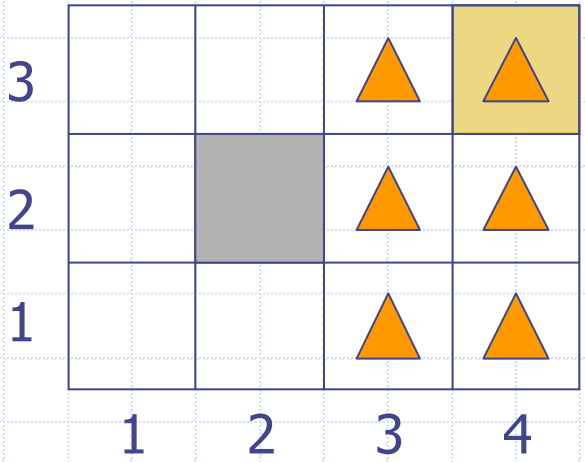
Sequence of Actions

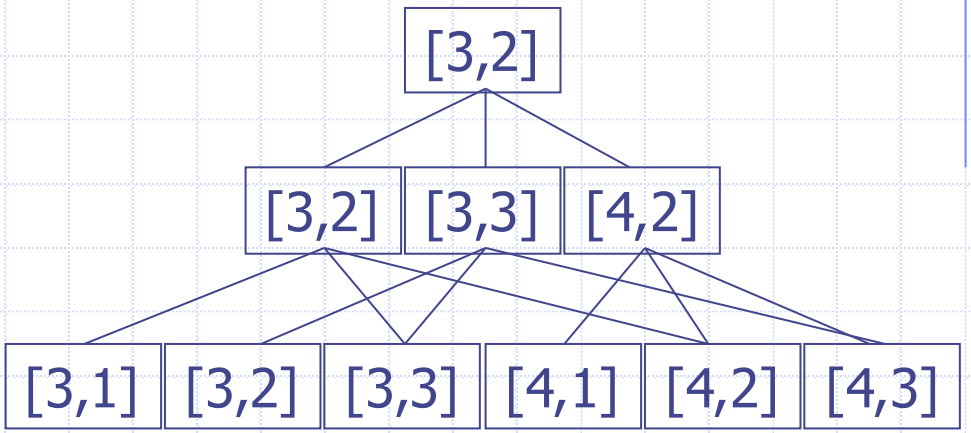




- Planned sequence of actions: (U, R)
- U is executed

Histories

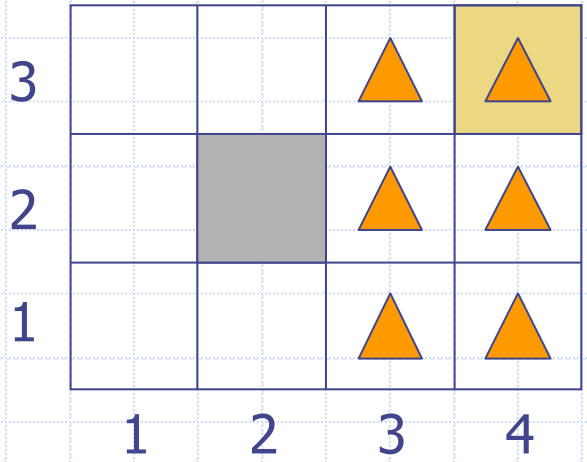




- Planned sequence of actions: (U, R)
- U has been executed
- R is executed

- There are 9 possible sequences of states – called histories – and 6 possible final states for the robot!

Probability of Reaching the Goal



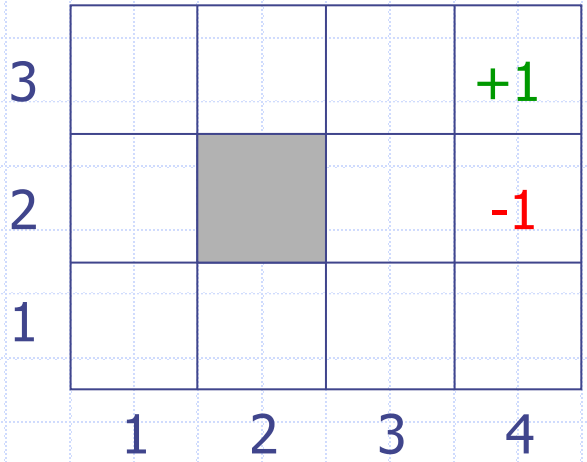
- **P**([4,3] | (U,R).[3,2]) =

  P([4,3] | R.[3,3]) x **P**([3,3] | U.[3,2])
  + **P**([4,3] | U.[4,2]) x **P**([4,2] | R.[3,2])

- **P**([4,3] | R.[3,3]) = 0.8
- **P**([4,3] | U.[4,2]) = 0.1
- **P**([3,3] | U.[3,2]) = 0.8
- **P**([4,2] | R.[3,2]) = 0.1

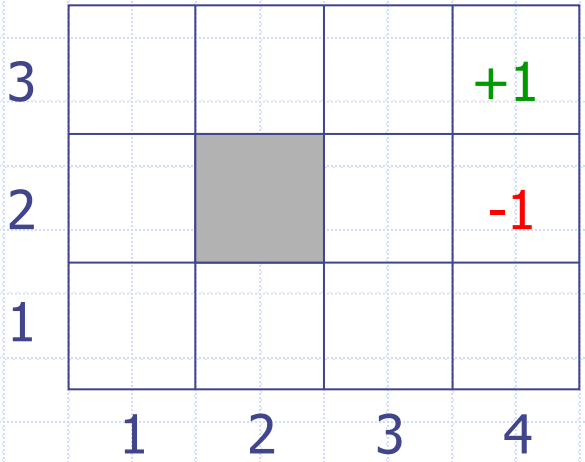
- **P**([4,3] | (U,R).[3,2]) = (0.64 + 0.01) = 0.65

Utility Function



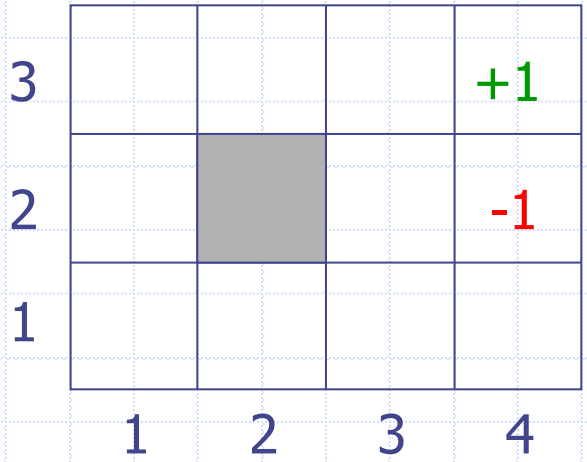
- [4,3] provides power supply
- [4,2] is a sand area from which the robot cannot escape

Utility Function



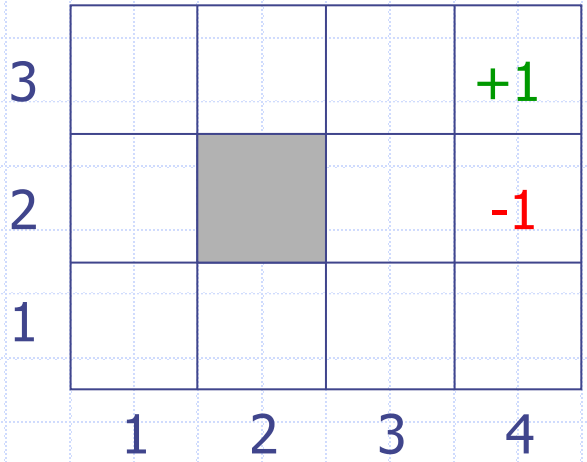
- [4,3] provides power supply
- [4,2] is a sand area from which the robot cannot escape
- The robot needs to recharge its batteries

Utility Function



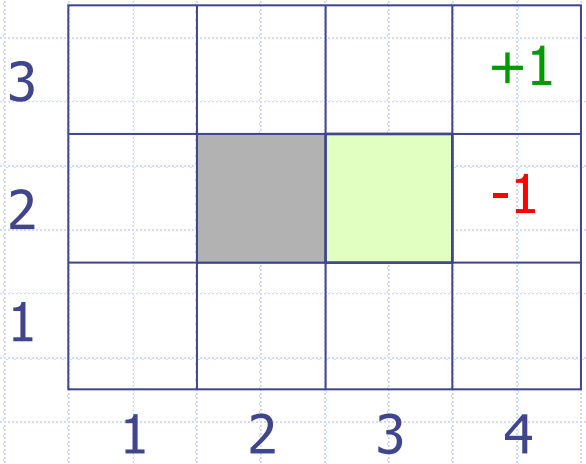
- [4,3] provides power supply
- [4,2] is a sand area from which the robot cannot escape
- The robot needs to recharge its batteries
- [4,3] or [4,2] are terminal states

Utility of a History



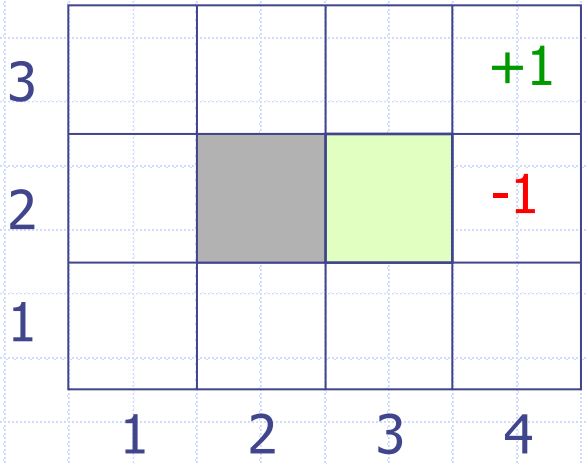
- [4,3] provides power supply
- [4,2] is a sand area from which the robot cannot escape
- The robot needs to recharge its batteries
- [4,3] or [4,2] are terminal states
- The utility of a history is defined by the utility of the last state (+1 or –1) minus n/25, where n is the number of moves

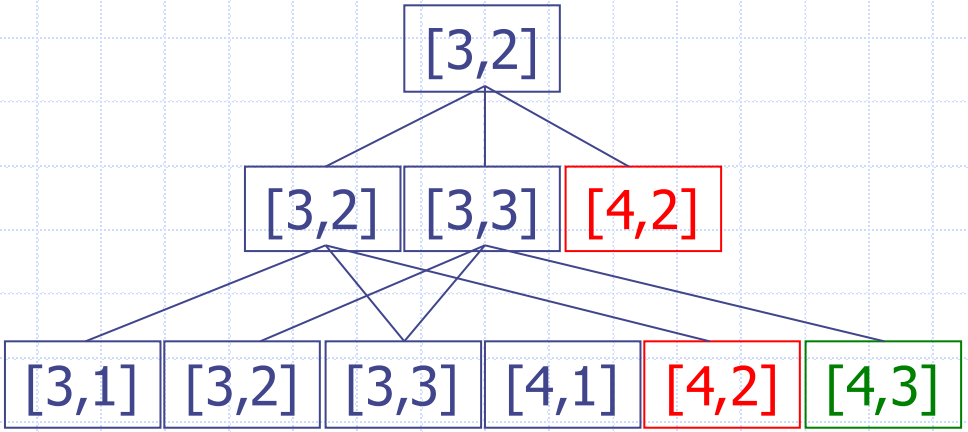
Utility of an Action Sequence



- Consider the action sequence (U,R) from [3,2]

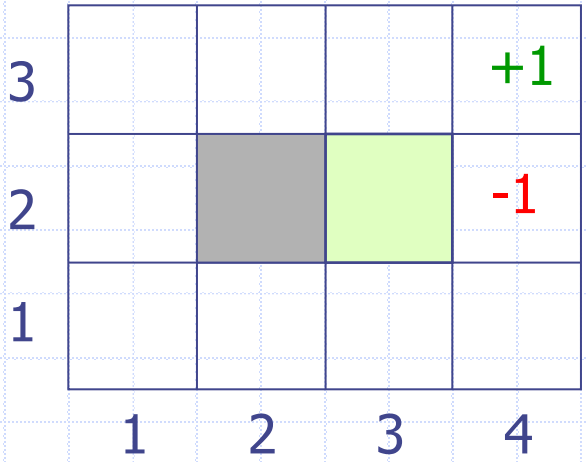
Utility of an Action Sequence

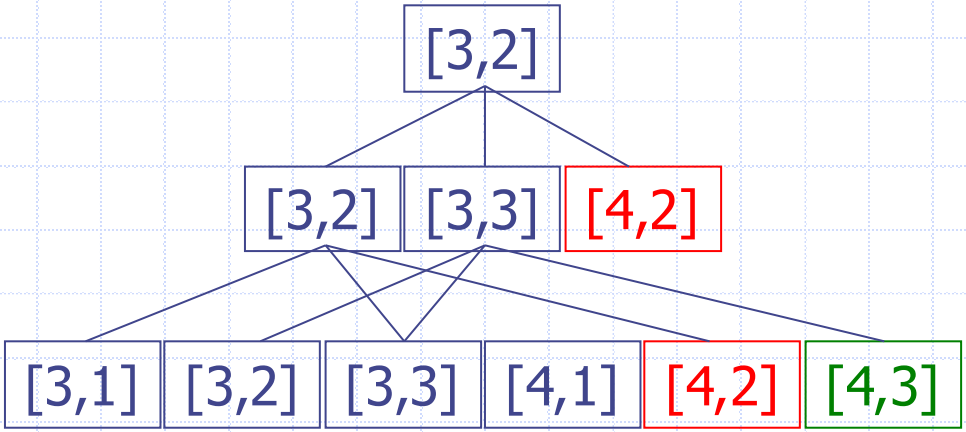




- Consider the action sequence (U,R) from [3,2]
- A run produces one among 7 possible histories, each with some probability

Utility of an Action Sequence

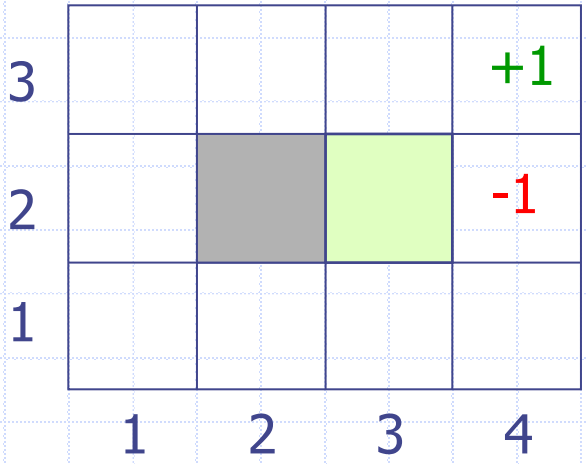


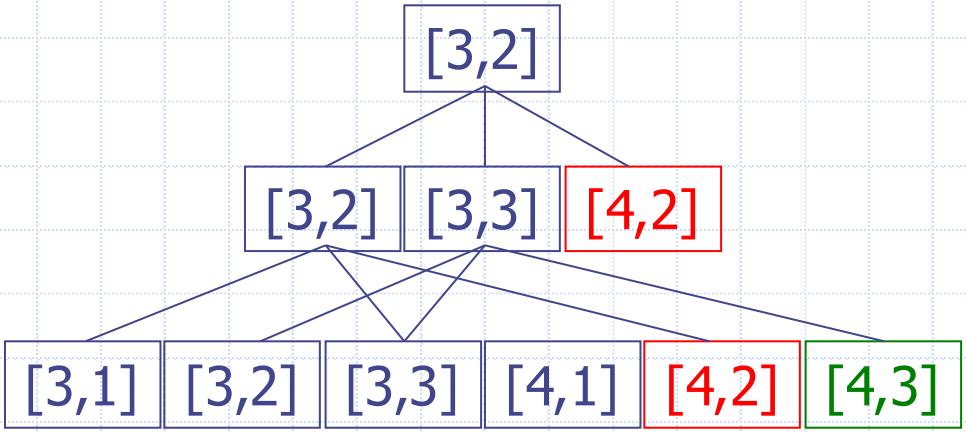


- Consider the action sequence (U,R) from [3,2]
- A run produces one among 7 possible histories, each with some probability
- The utility of the sequence is the expected utility of the histories:

$$U = \Sigma_h U_h \mathbf{P}(h)$$

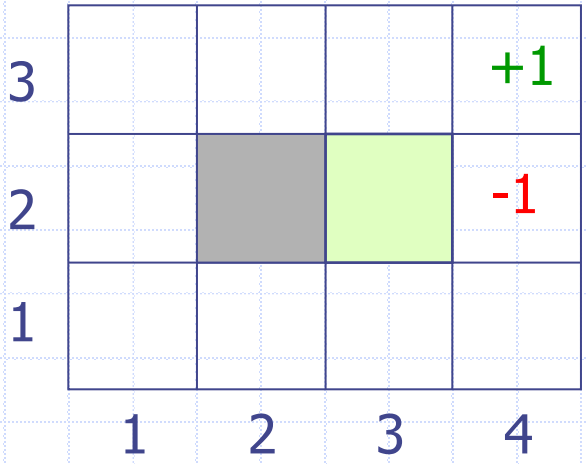
Optimal Action Sequence

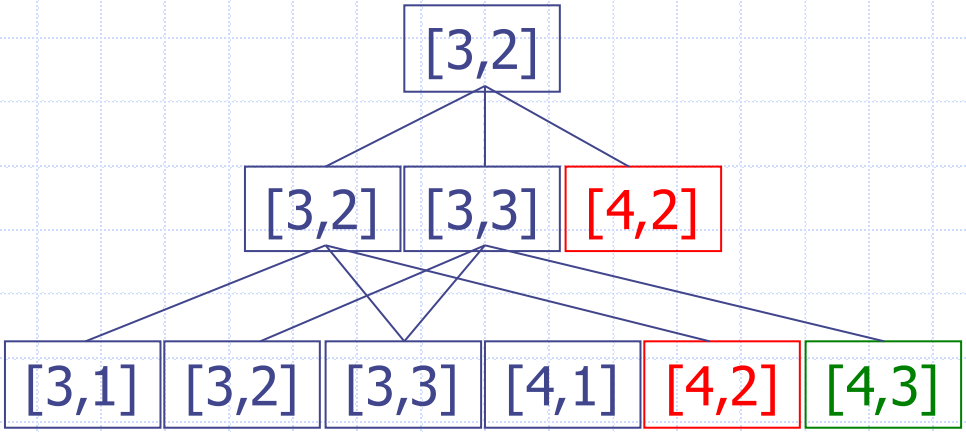




- Consider the action sequence (U,R) from [3,2]
- A run produces one among 7 possible histories, each with some probability
- The utility of the sequence is the expected utility of the histories
- The optimal sequence is the one with maximal utility

Optimal Action Sequence





- Consider the action sequence (U,R) from [3,2]
- A run produces one among 7 possible histories, each with some probability
- The utility of the sequence is the expected utility of the histories
- The optimal sequence is the one with maximal utility
- **But is the optimal action sequence what we want to compute?**

only if the sequence is executed blindly!

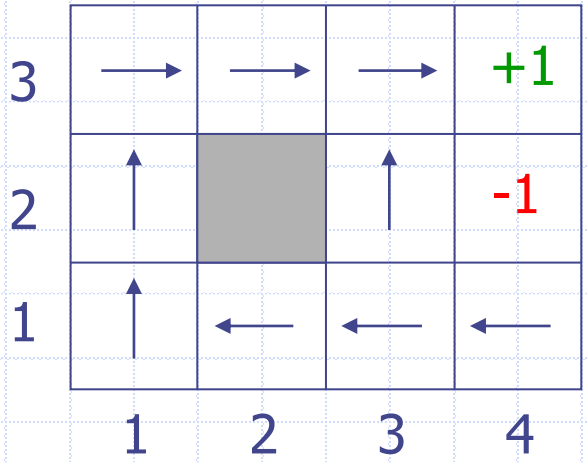
Reactive Agent Algorithm

Repeat:

- s ← sensed state
- If s is terminal then exit
- a ← choose action (given s)
- Perform a

Accessible or observable state

Policy (Reactive/Closed-Loop Strategy)



- A policy Π is a complete mapping from states to actions

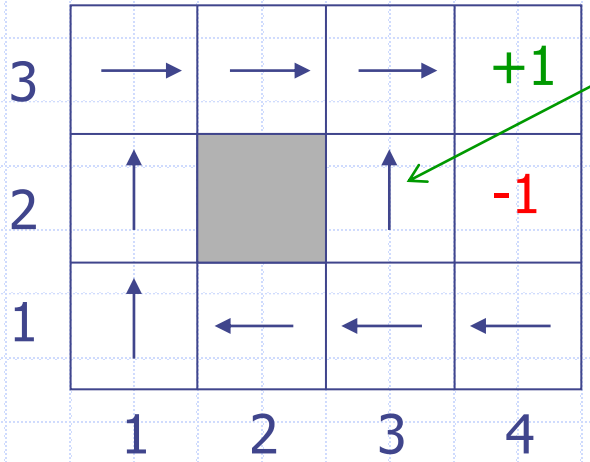
Reactive Agent Algorithm

Repeat:

- s ← sensed state
- If s is terminal then exit
- a ← Π(s)
- Perform a

Optimal Policy

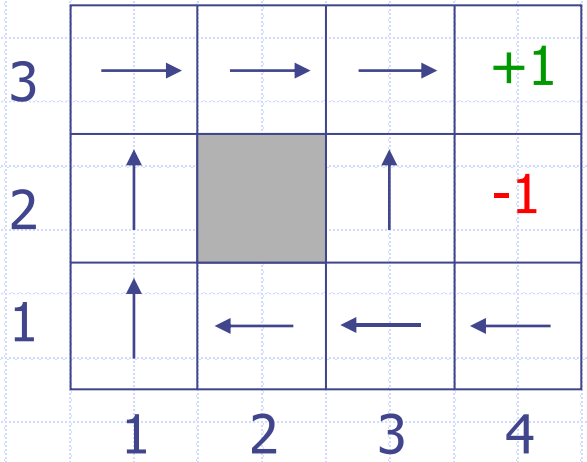
Note that [3,2] is a "dangerous" state that the optimal policy tries to avoid



- A policy Π is a complete mapping from states to actions
- The optimal policy Π^* is the one that always yields a history (ending at a terminal state) with maximal expected utility

Makes sense because of Markov property

Optimal Policy



- A policy Π is a complete mapping from states to actions
- The optimal policy Π^* is the one that always yields a history with maximal expected utility

How to compute Π^*?

This problem is called a Markov Decision Problem (MDP)

Additive Utility

- History $H = (s_0, s_1, \ldots, s_n)$
- The utility of H is additive iff:

$$\mathcal{U}(s_0, s_1, \ldots, s_n) = \mathcal{R}(0) + \mathcal{U}(s_1, \ldots, s_n) = \sum \mathcal{R}(i)$$

Reward

Additive Utility

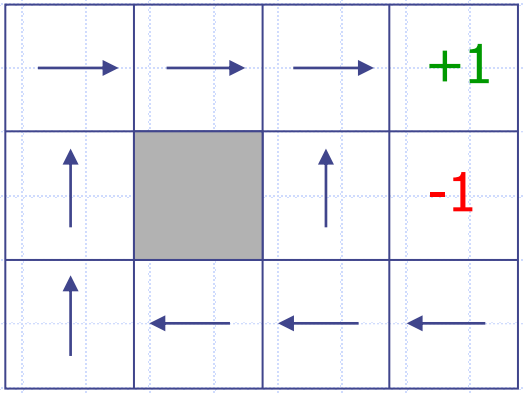
- History $H = (s_0, s_1, \ldots, s_n)$
- The utility of H is additive iff:

$$\mathcal{U}(s_0, s_1, \ldots, s_n) = \mathcal{R}(0) + \mathcal{U}(s_1, \ldots, s_n) = \sum \mathcal{R}(i)$$

- Robot navigation example:
  - $\mathcal{R}(n) = +1$ if $s_n = [4,3]$
  - $\mathcal{R}(n) = -1$ if $s_n = [4,2]$
  - $\mathcal{R}(i) = -1/25$ if $i = 0, \ldots, n-1$

Principle of Max Expected Utility

- History $H = (s_0, s_1, \ldots, s_n)$
- Utility of H: $\mathcal{U}(s_0, s_1, \ldots, s_n) = \sum \mathcal{R}(i)$



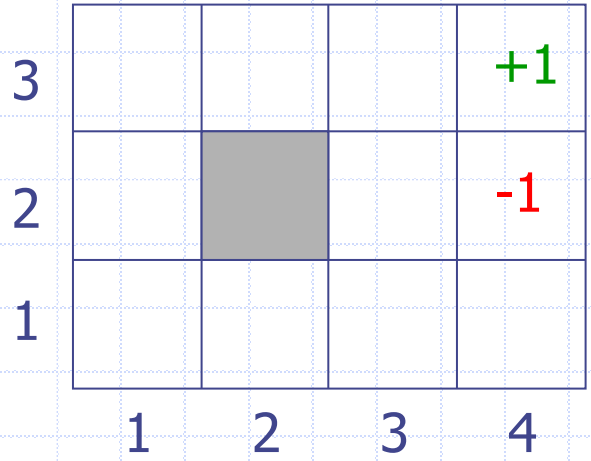
First-step analysis →

- $\mathcal{U}(i) = \mathcal{R}(i) + \max_a \sum_k \mathbf{P}(k \mid a.i)\, \mathcal{U}(k)$
- $\Pi^*(i) = \arg\max_a \sum_k \mathbf{P}(k \mid a.i)\, \mathcal{U}(k)$

Value Iteration

- Initialize the utility of each non-terminal state s_i to $\mathcal{U}_0(i) = 0$
- For t = 0, 1, 2, …, do:

$$\mathcal{U}_{t+1}(i) \leftarrow \mathcal{R}(i) + \max_a \sum_k \mathbf{P}(k \mid a.i)\, \mathcal{U}_t(k)$$



Uncertainty

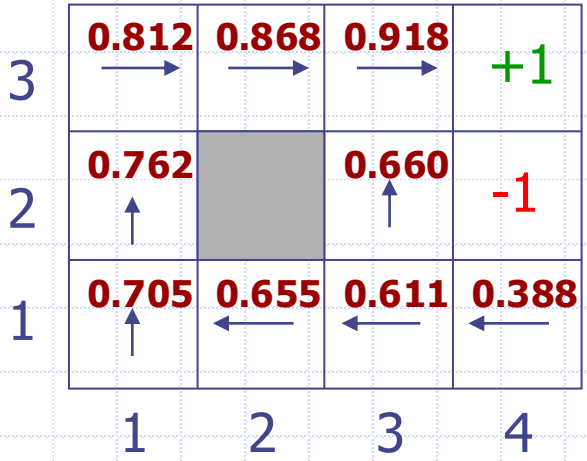
Actions	Up	Left	Right	Down
Up	0.8	0.1	0.1	0.0
Left	0.1	0.9	0.0	0.0
Right	0.1	0.0	0.8	0.1
Down	0.0	0.0	0.1	0.9

Value Iteration

Note the importance of terminal states and connectivity of the state-transition graph

- Initialize the utility of each non-terminal state s_i to $\mathcal{U}_0(i) = 0$
- For t = 0, 1, 2, …, do:

$$\mathcal{U}_{t+1}(i) \leftarrow \mathcal{R}(i) + \max_a \sum_k \mathbf{P}(k \mid a.i)\, \mathcal{U}_t(k)$$





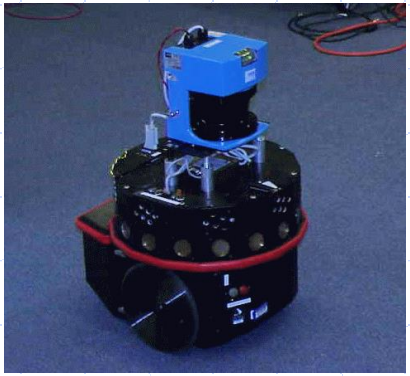
Policy Iteration

- Pick a policy Π at random
- Repeat:
  - Compute the utility of each state for Π

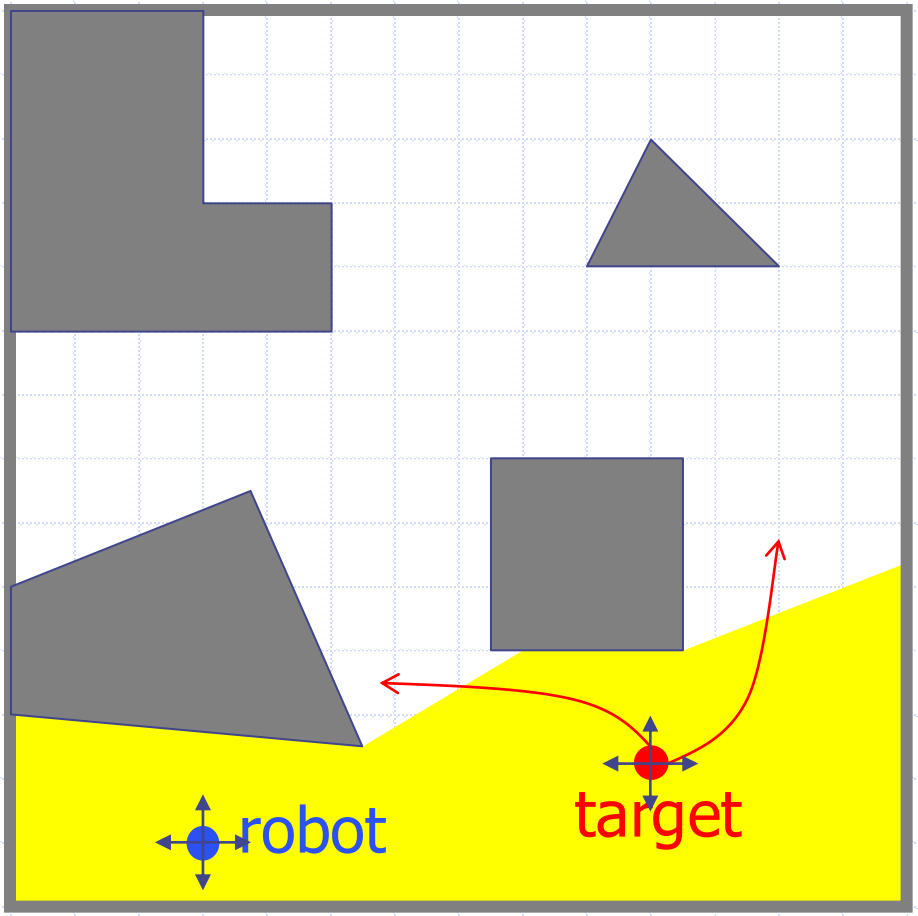
    $\mathcal{U}_{t+1}(i) \leftarrow \mathcal{R}(i) + \sum_k \mathbf{P}(k \mid \Pi(i).i)\ \mathcal{U}_t(k)$
  - Compute the policy Π' given these utilities

    $\Pi'(i) = \arg\max_a \sum_k \mathbf{P}(k \mid a.i)\ \mathcal{U}(k)$
  - If $\Pi' = \Pi$ then return Π

Example: Tracking a Target



- The robot must keep the target in view
- The target's trajectory is not known in advance



Example: Tracking a Target

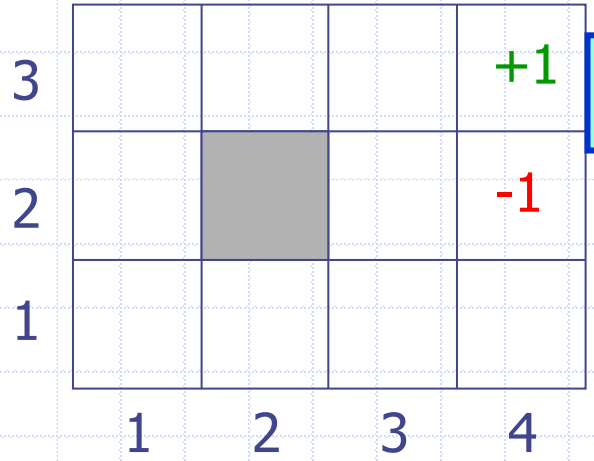


Example: Tracking a Target



Infinite Horizon

In many problems, e.g., the robot navigation example, histories are potentially unbounded and the same state can be reached many times



What if the robot lives forever?

One trick:
Use discounting to make infinite Horizon problem mathematically tractible

POMDP (Partially Observable Markov Decision Problem)

- A sensing operation returns multiple states, with a probability distribution
- Choosing the action that maximizes the expected utility of this state distribution assuming "state utilities" computed as above is not good enough, and actually does not make sense (is not rational)

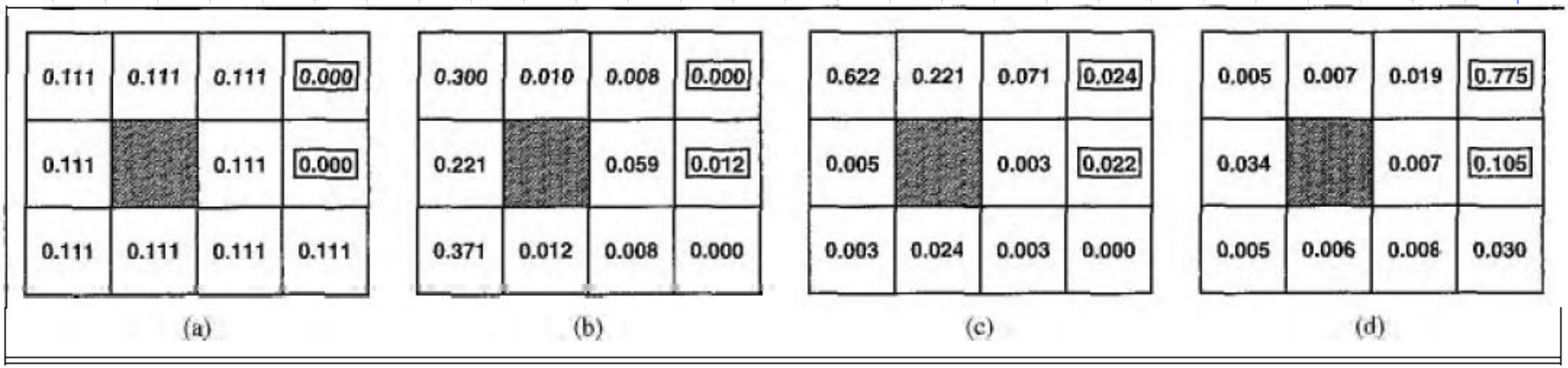


Figure 17.8 (a) The initial probability distribution for the agent's location. (b) After moving *Left* five times. (c) After moving Up five times. (d) After moving Right five times.

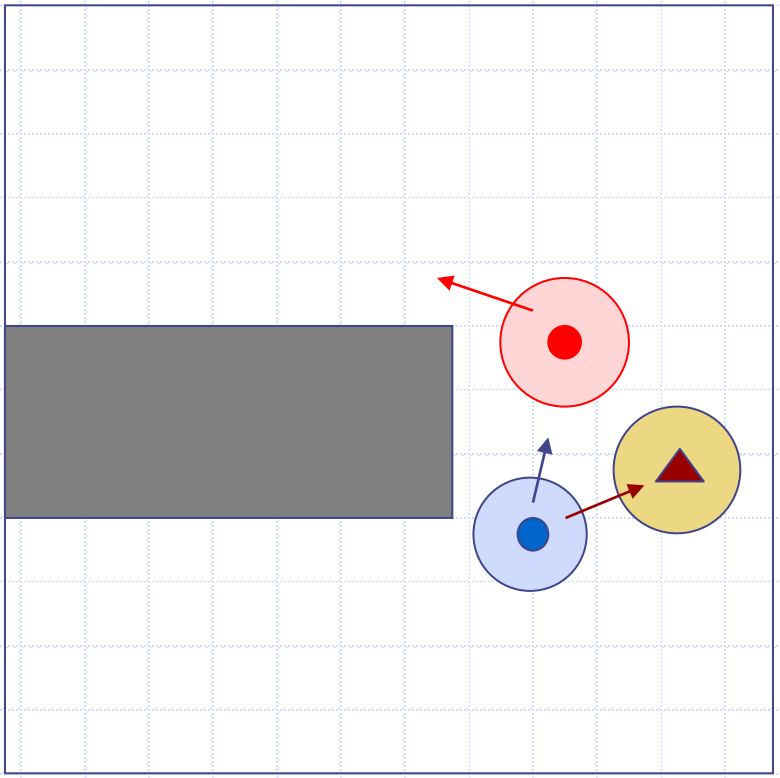
- The agent does not necessarily know which state it is in, so it cannot execute the action $\pi(s)$ recommended for that state.
- In POMDPs, the concept is refined somewhat. A belief state b is now a probability distribution over all possible states.
- The optimal action depends only on the agent's current belief state.
- That is, the optimal policy can be described by a mapping $\pi^*(b)$ from belief states to actions.
- It does not depend on the actual state the agent is in. Because the agent does not know its actual state; all it knows is the belief state.

Example: Target Tracking

There is uncertainty in the robot's and target's positions, and this uncertainty grows with further motion

There is a risk that the target escape behind the corner requiring the robot to move appropriately

But there is a positioning landmark nearby. Should the robot tries to reduce position uncertainty?



Summary

- Decision making under uncertainty
- Utility function
- Optimal policy
- Maximal expected utility
- Value iteration
- Policy iteration